

The full twist is not generated by its square

Disproof of conjecture 00000000637

ColdSmile — 5 October 2026

Result. At the allowed strand number $n = 3$, the full twist T is a central pure braid but does not belong to $\langle T^2 \rangle$. This disproves the center assertion and hence the conjecture's conjunction. No claim about stable commutator length is needed or proved.

1. Original statement and precise scope

The original file is reproduced byte for byte as `original-conjecture.md`. Its two versions, with mathematical notation typeset, are:

English. Definition: The center $Z(P_n)$ of the pure braid group. Conjecture: For $n \geq 3$, $Z(P_n)$ is generated by the square of the full twist, and the stable commutator length of P_n/Z is bounded (combining the center with scl).

中文. 定义: 纯辫中心 $Z(P_n)$ 。猜想: P_n 的中心由全 twist 的平方生成 ($n \geq 3$), 且商 P_n/Z 的稳定交换子长度有界 (中心与 scl 的组合)。

We use the standard Artin braid groups of the disk and their pure subgroups. For three strands, put

$$B_3 = \langle a, b \mid aba = bab \rangle, \quad \pi : B_3 \longrightarrow S_3, \quad a \mapsto (0\ 1), \quad b \mapsto (1\ 2), \quad P_3 = \ker \pi.$$

This is the standard algebraic presentation and permutation-kernel definition [1, 2]; labels 0, 1, 2 replace the usual 1, 2, 3. There are no distant-generator relations at three strands. The center means the actual subgroup of elements commuting with every pure braid, and $\langle u \rangle = \{u^k : k \in \mathbb{Z}\}$.

Write $\Delta = aba$ for the half twist and $T = (ab)^3 = \Delta^2$ for the full twist, following the standard convention [3]. Thus the literal square in both source languages is $T^2 = \Delta^4$. The equality $T = \Delta^2$ is proved below and in Lean; it is not an extra assumption. This submission does not replace “full twist” by “half twist”.

The first conjunct has the form $C(n) : Z(P_n) = \langle T_n^2 \rangle$. The universally quantified conjunction necessarily implies $C(3)$. In the second conjunct, Z naturally denotes $Z(P_n)$; the source does not specify the scl domain or whether the bound is uniform in n . Our argument is independent of these choices: for every proposition Q , $\neg C(3)$ implies $\neg(C(3) \wedge Q)$. The parameter Q in the Lean conjunction theorem is this logical placeholder, not a definition of scl or of the entire family of braid groups.

2. Purity and centrality of the full twist

The transpositions $(0\ 1)$ and $(1\ 2)$ satisfy the braid relation, so π is well defined. Their product is a three-cycle, hence $\pi((ab)^3) = 1$. Thus $T \in P_3$.

The Artin relation gives

$$T = (ab)^3 = (aba)(bab) = (aba)(aba) = \Delta^2, \quad \Delta a = b\Delta, \quad \Delta b = a\Delta.$$

Indeed, $\Delta a = (aba)a = (bab)a = b(aba)$ and $\Delta b = (aba)b = a(bab) = a(aba)$. Consequently $\Delta^2 a = a\Delta^2$ and $\Delta^2 b = b\Delta^2$. Since a, b generate B_3 , their centralizer condition implies that T commutes with every element of B_3 . Restriction to the subgroup P_3 therefore proves $T \in Z(P_3)$.

3. The exponent homomorphism separates the square subgroup

Send each of a, b to $1 \in \mathbb{Z}$. Both sides of $aba = bab$ have exponent sum 3, so this induces a well-defined group homomorphism

$$e : B_3 \longrightarrow (\mathbb{Z}, +), \quad e(a) = e(b) = 1.$$

In particular $e(T) = 6$ and, for every integer k ,

$$e((T^2)^k) = k e(T^2) = 12k.$$

If $T = (T^2)^k$, then $6 = 12k$. No integer satisfies this equation: it would imply $1 = 2k$. Therefore $T \notin \langle T^2 \rangle$, with negative powers included. This argument does not require a structure theorem for P_3 or an assumed description of its center.

Counterexample. *With the standard definitions above,*

$$T \in Z(P_3) \setminus \langle T^2 \rangle, \quad Z(P_3) \neq \langle T^2 \rangle.$$

Thus conjecture 00000000637, as written in both languages, is false.

Proof. Section 2 gives membership in the center; the exponent calculation gives nonmembership in the proposed generating subgroup. Since $3 \geq 3$, the universal source statement includes this false center equality. A conjunction with a false first conjunct is false, whatever its second conjunct. $\square$

4. Correspondence with the formalization

All mathematical declarations lie in namespace `Conjecture637` of `Conjecture637.lean`. They use the standard Mathlib definitions, not custom replacements for center or integer powers.

Mathematical object	Formal construction
B_3 and a, b	<code>BraidThree</code> is <code>PresentedGroup</code> <code>braidRelations</code> : free group on <code>Bool</code> , modulo the normal closure of the single relator $(aba)(bab)^{-1}$. <code>sigmaOne</code> , <code>sigmaTwo</code> are its generators.
π and P_3	<code>strandPermutation</code> is built by the presentation's universal property after proving the relation. <code>pureBraidSubgroup</code> is its kernel; <code>PureBraidThree</code> is that subgroup as a group.
Δ, T	<code>halfTwist</code> , <code>fullTwist</code> have the displayed words. <code>pureFullTwist</code> pairs T with the proved purity witness.
Center and $\langle T^2 \rangle$	Mathlib's <code>Subgroup.center</code> and <code>Subgroup.zpowers</code> , in the actual subgroup <code>PureBraidThree</code> .
e	<code>exponentSum</code> takes values in <code>Multiplicative ℤ</code> , the additive group written multiplicatively; the relation and generator values are proved before its use.

The proof proceeds through `fullTwist_eq_halfTwist_sq`, the two generator commutation lemmas, `fullTwist_mem_center`, `fullTwist_is_pure`, and `pureFullTwist_mem_center`. Generation of the entire presented group is used to extend commutation beyond the generators. The exponent obstruction is proved in `fullTwist_not_zpower_square`; subtype coercion then yields `pureFullTwist_not_mem_square_zpowers`.

The decisive theorem is `center_not_generated_by_square_of_fullTwist`. The theorem `three_is_in_source_range` checks the admissible strand number. Finally, `required_conjunction_at_three_false` proves the negation of `RequiredConjunctionAtThree Q` for arbitrary $Q : \text{Prop}$. No theorem about `scl` and no full formalization for every strand number is claimed.

5. Verification, reproducibility, and trust scope

The project pins Lean 4.19.0 and Mathlib v4.19.0, whose exact Git revision is `c44e0c8ee63ca166450922a373c7409c5d26b00b`. All nine dependency revisions are fixed in `lake-manifest.json`. The actual compiler commit was checked as `6caaee842e9495688c1567e78c0e68dbb96942aa`.

The author and an independent reviewer executed the supplied `verify.py` (22 subprocesses per run). Root and reviewer also built frozen source in separate fresh directories, replayed `Conjecture637.lean`, `Audit.lean`, and `Check.lean` with warnings treated as errors, and checked dependencies before and after execution. All final runs passed. Existing pinned Mathlib compiled dependencies were reused; the submitted project’s own build products were absent before each fresh build.

The 36 handwritten declarations comprise 15 definitions/abbreviations and 21 theorems, with no handwritten instances. `Audit.lean` prints axiom dependencies for all 36; `Check.lean` also prints definitions and theorem types. A separate inspection harness inventories every compiled declaration originating in the implementation modules, including generated declarations. It produces types, safety flags, axiom sets, and transitive dependency checks.

That complete inventory has 194 entries: 81 safe logical declarations, 90 unsafe compiler runtime definitions, and 23 unsafe specialization axiom stubs generated by Lean. The first root and reviewer inspection runs rejected these unclassified stubs. Both failures are retained. The subsequent exact classification is justified by the pinned Lean compiler’s specialization code [4] and its proof-erasure marker documentation [5]. Every stub’s name and unsafe dependency closure is explicitly checked; no blanket extra-axiom exemption is used. All 194 entries remain in the records.

For every one of the 81 safe logical declarations, the transitive closure contains no unsafe, partial, or missing dependency, and its axiom set is a subset of `propext`, `Classical.choice`, and `Quot.sound`. Thus no logical proof depends on the runtime stubs or proof-erasure markers. There are no admitted proofs, author-added axioms, or `native_decide` uses. The finite permutation checks use ordinary kernel-checked `decide`. Group normalization and integer arithmetic likewise produce checked proof terms. No mathematical computation is performed by an external auxiliary program. Python files only orchestrate execution and inspect verification evidence.

Reproduction. With the pinned toolchain and packages available, run `lake build`, then `python3 verify.py`; replay `Check.lean` with `lake env lean -DwarningAsError=true Check.lean`. The submission README gives the full independent inspector invocation, frozen hashes, dependency setup, and the locations of successful and failed logs. Inspection metaprogramming is outside the mathematical proof and asserts no theorem.

Review status. These are local build and independent semantic checks. Maintainer review and competition acceptance remain separate decisions. The independent review record covers this report and its matching PDF. Definitions-only primary references were used to fix braid terminology; the formal argument proves the required group identities directly. A source lookup helper did not return a complete search ledger, so the provenance record does not claim that no incidental mathematical results were exposed.

References

- [1] I. Tuba and H. Wenzl, *Representations of the braid group B_3 and of $SL(2, \mathbb{Z})$* , Pacific J. Math. 197 (2001), §1.1, p.492. [Primary source](#).
- [2] J. González-Meneses, *Basic results on braid groups*, Ann. Math. Blaise Pascal 18 (2011), §2.1, p.26, equation (2.1). [Primary source](#).
- [3] J. S. Birman, V. Gebhardt and J. González-Meneses, *Conjugacy in Garside groups III: Periodic braids*, J. Algebra 316 (2007), §2, p.748. [Primary source](#).
- [4] Lean 4.19.0, `specialize.cpp`, lines 906–920. [Unsafe specialization stubs](#).
- [5] Lean 4.19.0, `Init/Prelude.lean`, lines 129–137. [Compiler proof-erasure marker](#).